Inteligência Artificial

Busca

Prof.

Fernando H Carvalho Silva

fernando.henrique@puc-campinas.edu.br

Sumário

1 Introdução

2 Busca Não-Informada (Força Bruta)

Introdução

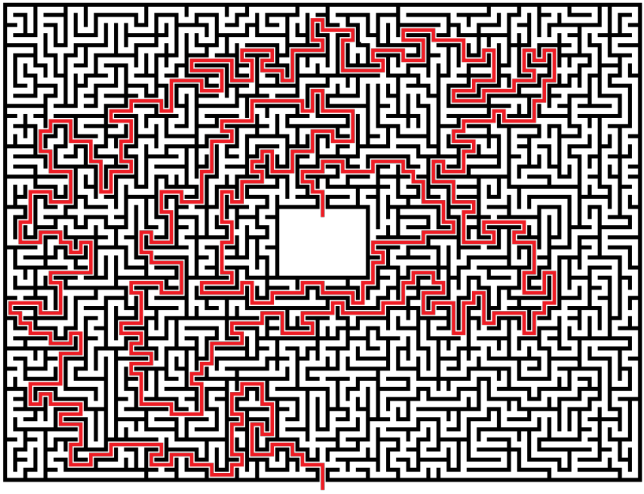
1 Introdução

- Tarefa de Busca

2 Busca Não-Informada (Força Bruta)

Tarefa de Busca

Busca



Motivação

Foco em atingir um objetivo, por exemplo:

- Saída de um labirinto
- Formas de colorir um mapa
- Melhor jogada a ser feita em um jogo
- Melhor caminho a ser explorado

Características importantes

Precisamos avaliar algumas coisas importantes em algoritmos de busca:

- Completude: O algoritmo encontra uma solução, dado que ela exista?
- Otimalidade: Quando o algoritmo encontra uma solução, ela é a melhor possível?
- Complexidade de Tempo: Como o tempo de processamento do algoritmo se comporta em função do tamanho da entrada?
- Complexidade de Espaço: Como o espaço ocupado pelas informações necessárias ao algoritmo se comporta em função do tamanho da entrada?

Funcionamento

A busca visa explorar as várias possibilidades de solução, para isso são necessários:

- Estados
- Estado inicial
- Função sucessora
- Avaliação de estado/solução encontrado(a)

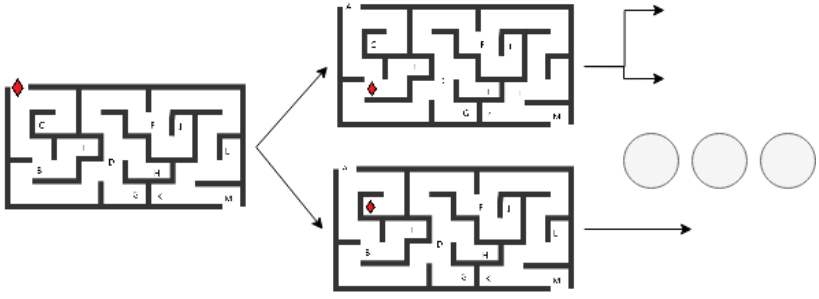
Estados

Uma das possíveis configurações do meu problema, por exemplo:

- Posição das peças em um jogo de xadrez
- Cores atribuídas às regiões de um mapa
- Caminho percorrido até agora

Função Sucessora

As possíveis transições entre estados podem ser representadas e armazenados em um grafo (ou árvore), onde cada nó representa um estado e cada aresta representa uma transição



Solução

Definição

Uma solução correta é um conjunto de passos que leva de um estado inicial ao estado objetivo

Tarefas de busca podem se preocupar com dois tipos de problemas:

- Planejamento: O caminho que leva a solução é importante (e.g. labirinto, rota)
- Identificação: Saber/Encontrar a solução é o importante (e.g. colorir um mapa)

Busca Não-Informada (Força Bruta)

1 Introdução

2 Busca Não-Informada (Força Bruta)

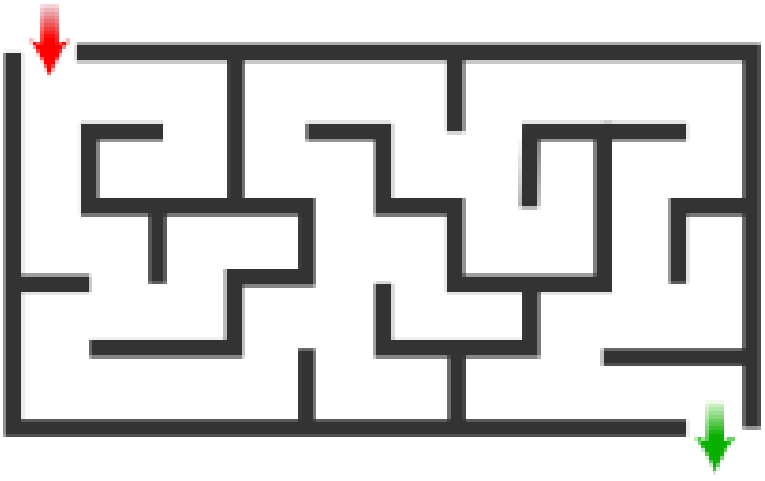
- Exemplos de Problemas
- Efetuando a Busca
- Busca em Largura
- Busca em Profundidade
- Busca em Profundidade Limitada (Iterativa)

Características

Na busca não-informada não temos informações que nos ajudem a afunilar o problema, sendo necessário buscar exaustivamente pelo espaço de estados.

Exemplos de Problemas

Labirinto



Labirinto

Concordam que não é importante guardar cada centímetro que nosso visitante percorre? Apenas as bifurcações e finais são importantes, pois são pontos de tomada de decisão

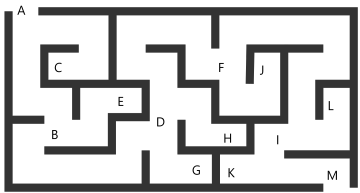


Figura 1: Podemos atribuir valores para cada bifurcação

Representando melhor o labirinto

As paredes e até mesmo o formato do labirinto não precisam ser armazenados, apenas a transição entre os posíveis estados.

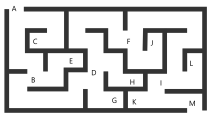


Figura 2: Representação visual

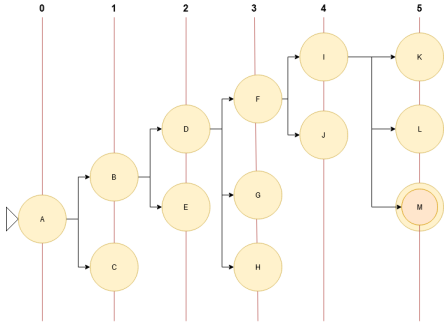


Figura 3: Como árvore. As linhas verticais indicam o nível; também marcamos os estados inicial (A, triangulo) e final (M, círculos concentricos)

8-Puzzle

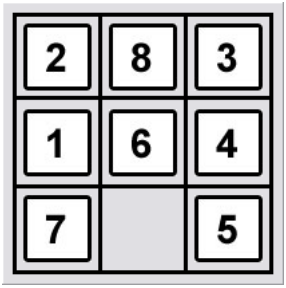


Figura 4: Ilustração do quebra-cabeças em um estado arbitrário

- As peças deslizam sobre o tablado
- O objetivo é deixar a o tabuleiro ordenado de 1 a 8 com o espaço em branco no final
- A priori 9! estados[a]

[a] 362.880

8-Puzzle

- Como podemos descrever um estado do quebra-cabeças?
- E sua função sucessora?
- E a avaliação?
- É um problema de Planejamento ou Identificação?

8-Puzzle - Estado e função sucessora computacionalmente 1 - Matriz

- Podemos deixar como uma matriz 3x3, com elementos a_{ij}
- Trocar a casa e branco (0 ou X) pode ser feito de 4 maneiras:
  - direita: casa é o índice i de X, mais 1
  - esquerda: casa é o índice i de X, menos 1
  - cima: casa é o índice j de X, menos 1
  - baixo: casa é o índice j de X, mais 1
- Precisamos nos preocupar com as transações nas bordas, pela forma matricial podemos verificar facilmente extrapolações.

8-Puzzle - Estado e função computacionalmente 2 - Vetor

- Podemos deixar como um vetor de 9 posições
- Trocar a casa e branco (0 ou X) pode ser feito de 4 maneiras:
  - direita: casa é o índice de X, mais 1
  - esquerda: casa é o índice de X, menos 1
  - cima: casa é o índice de X, menos 3
  - baixo: casa é o índice de X, mais 3
- Precisamos nos preocupar com as transações nas bordas

Coloração de Mapas

- Queremos colorir um mapa de tal modo que regiões adjacentes possuam cores diferentes
- Vamos assumir uma quantidade limitada de cores. Como modelar esse problema?
- Estados e Função Sucessora?
- Avaliação de estado?

Efetuando a Busca

Busca

- Temos o conjunto de estados, as transições e a meta
- Como navegar sistematicamente pelo espaço de busca?
- Como podemos representar e explorar esse espaço?

Grafos

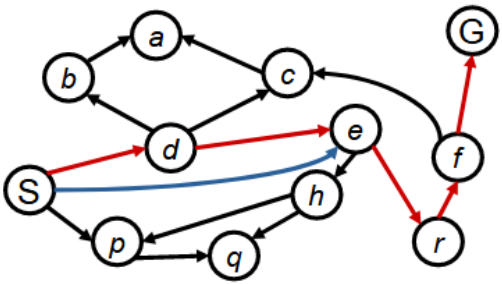


Figura 5: Espaço de estados representado como grafo. **Fonte**: imagem criada por Dan Klein e Pieter Abbeel para *CS188 Intro to AI* em *UC Berkeley*, disponível em http://ai.berkeley.edu

- Cada nó representa um estado
- São permitidas conexões entre quaisquer dois estados
- Cada estado está presente apenas uma vez

Árvore

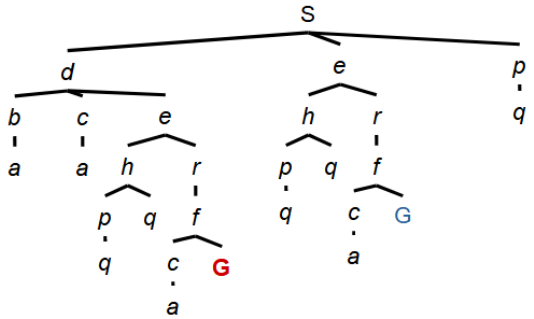


Figura 6: Espaço de estados representado como árvore. **Fonte**: adaptado de imagem criada por Dan Klein e Pieter Abbeel para *CS188 Intro to AI* em *UC Berkeley*, disponível em http://ai.berkeley.edu

- Cada nó representa um estado
- Mas também está relacionado ao caminho para o estado/solução
- Um mesmo estado pode estar presente mais de uma vez na árvore

Árvore e Grafo - Comparação (Geral)

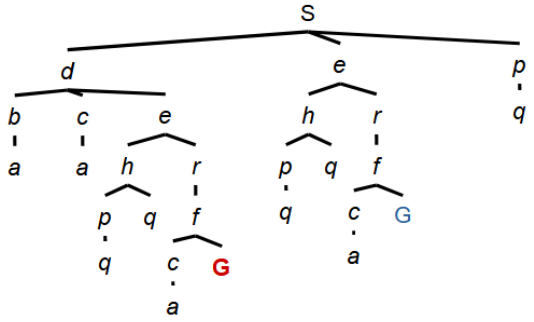


Figura 7: Espaço de estados representado como árvore. **Fonte**: adaptado de imagem criada por Dan Klein e Pieter Abbeel para *CS188 Intro to AI* em *UC Berkeley*, disponível em http://ai.berkeley.edu

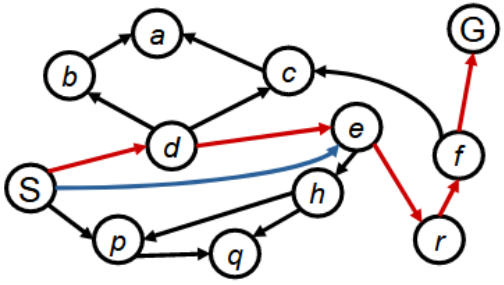
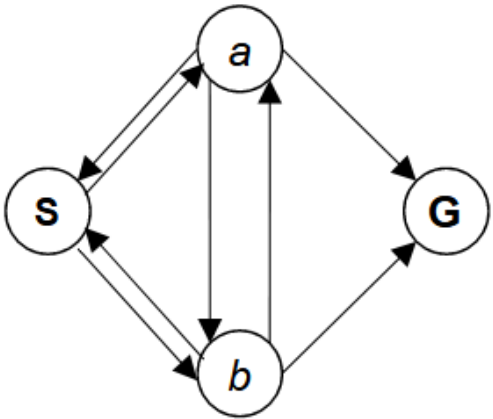


Figura 8: Espaço de estados representado como grafo. **Fonte**: adaptado de imagem criada por Dan Klein e Pieter Abbeel para *CS188 Intro to AI* em *UC Berkeley*, disponível em http://ai.berkeley.edu

Árvore e Grafo - Comparação (Laços)



- Como ficaria a árvore referente à esse grafo?
- A presença de um laço é importante? É importante re-visitar um estado/caminho?

Cuidados

Independente da estrutura, precisamos:

- Evitar laços
- Manter o mínimo de informação armazenada, o que muitas vezes implica em não gerar/armazenar todos os possíveis estados em memória

Escolhendo a ordem de visita

A busca em geral seguirá o seguinte algoritmo:

- Dado um estado inicial; enquanto ainda houverem nós para serem explorados:
- 1. Seleciona um nó não explorado
  2. Verificar se o nó atual é solução (T: retorna a solução)
  3. Adiciona novos nós em potencial seguindo estratégia
- Retorna: 'nenhuma solução encontrada'

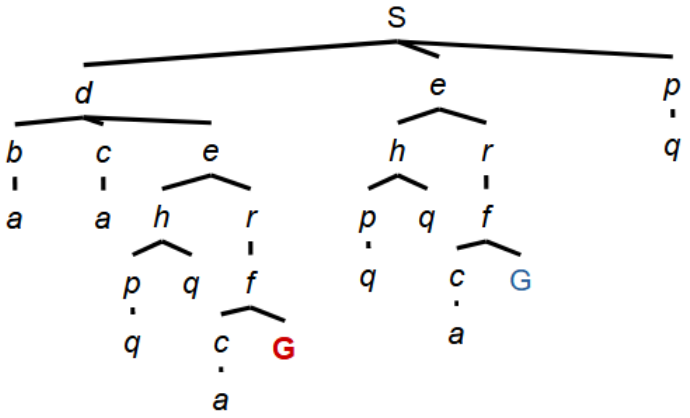
Escolhendo a ordem de visita

Possuímos 2 maneiras pricipais de lidar com as visitas:

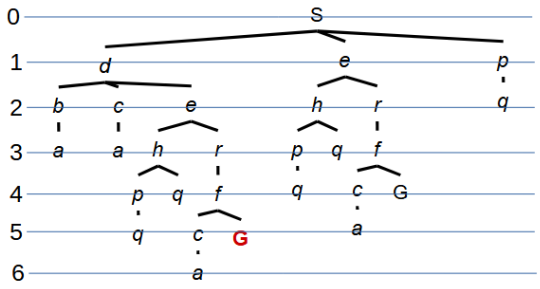
- Em largura *Breadth-First Search*
- Em profundidade *Depth-First Search*

Busca em Largura

Funcionamento



Funcionamento

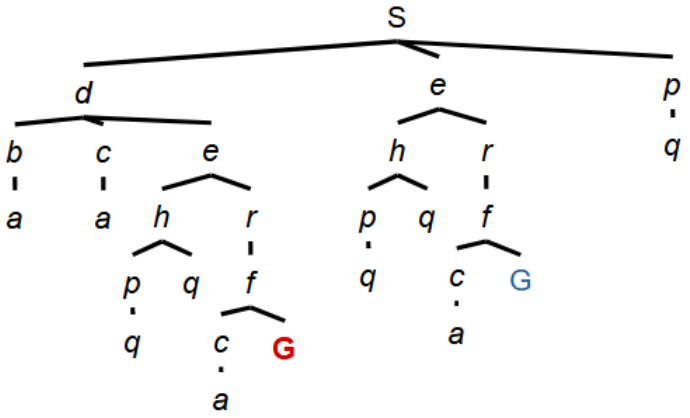


- Irá percorrer todos os nós em um nível antes de explorar os próximos níveis
- S, d, e, p, b, c, e, h, r, q ...

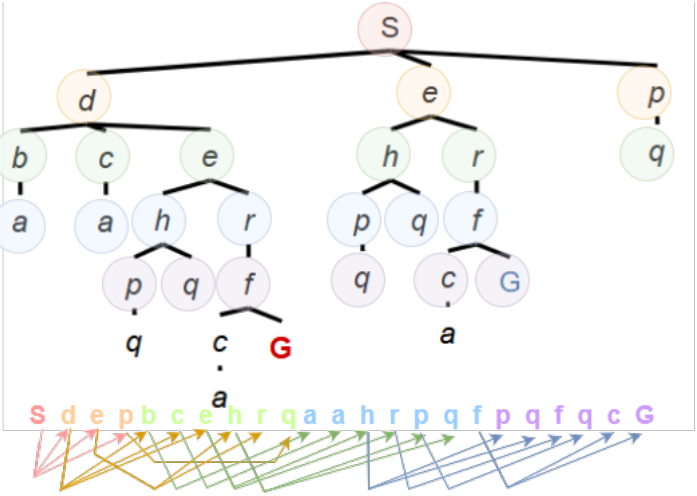
Implementação

- Utilizaremos uma estrutura de fila para implementar as visitas
- FIFO (First-In First-out)

Implementação usando fila - Lousa



Implementação usando fila - Referência

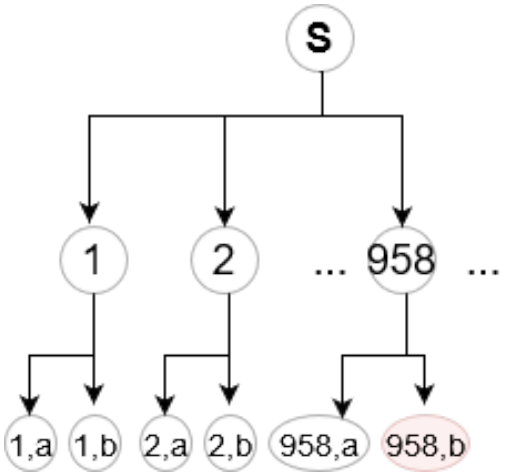


Características - Completude

- Se a solução existe, ela será encontrada, desde que a largura da árvore seja finita

Características - Completude

- Imaginem um problema de adivinhar um número inteiro, seguido de a ou b (imaginei <958,b>)
- Modelado da seguinte forma: explorarei as combinações de números e letras
- Como o nível dos números é infinito, a busca nunca irá terminar e atingir a solução
- Essa estratégia permite resolver usando busca em largura? É possível alterar a modelagem e resolver o problema com BL?

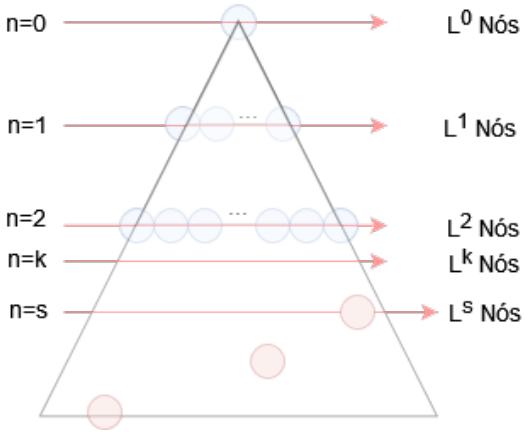


Características - Otimalidade

- Considerando que o nível da árvore está diretamente relacionado ao custo, tendo custos menores associados a níveis mais próximos da raiz da árvore
- A primeira solução encontrada será ótima

Características - Complexidade de Tempo

- Considerem o grau máximo de cada nó igual a L
- E que a solução no nível mais alto está no nível s (a contar de 0)
- A quantidade de nós em cada nível é igual a L^k sendo k o nível escolhido
- No pior caso a solução está no último nó do s-ésimo nível
- $O(\sum_{i=0}^{s} L^i) = O(L^s)$

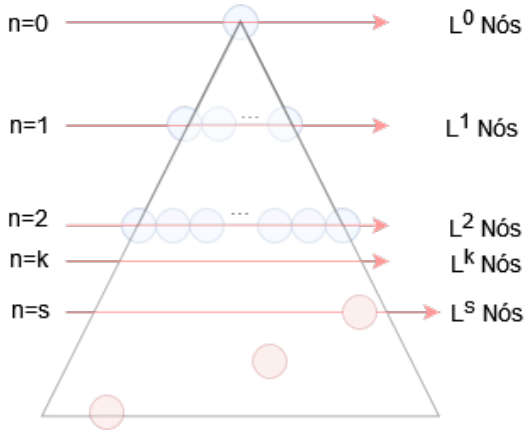


Características - Complexidade de Tempo

- Notem que no pior caso a busca irá percorrer toda a árvore
- Mas tende a operar melhor quando a solução está em níveis altos da árvore, além de árvores com largura menor

Características - Complexidade de Espaço

- Considerando que a árvore de decisão possui apenas os nós explorados
- $O(\sum_{i=0}^{s} L^i) = O(L^s)$ de forma análoga à complexidade de tempo

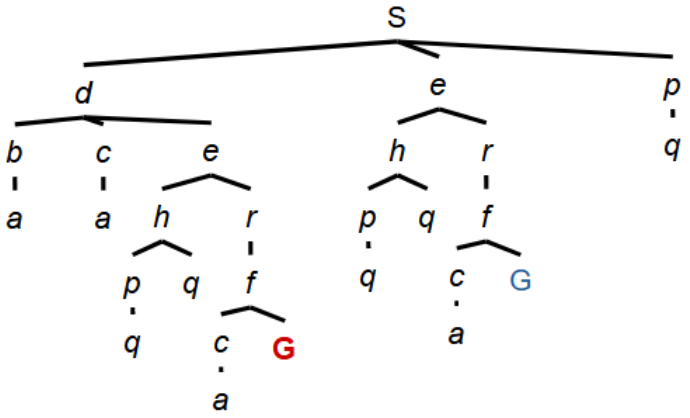


Uso

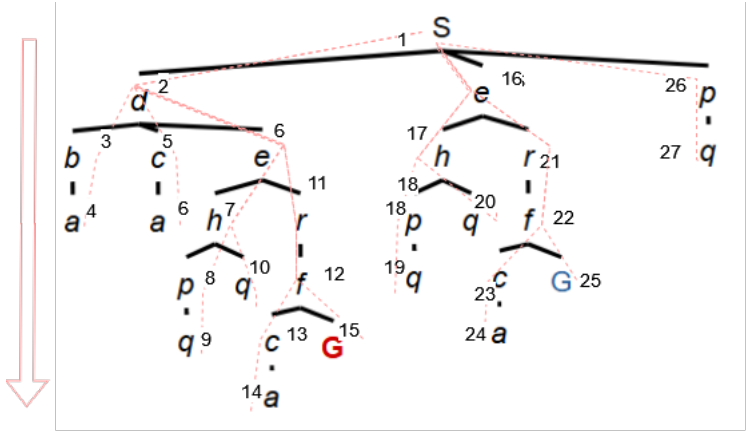
- Exemplos de problemas que parecem exigir menos tentativas seguindo essa estratégia?

Busca em Profundidade

Funcionamento



Funcionamento



Implementação

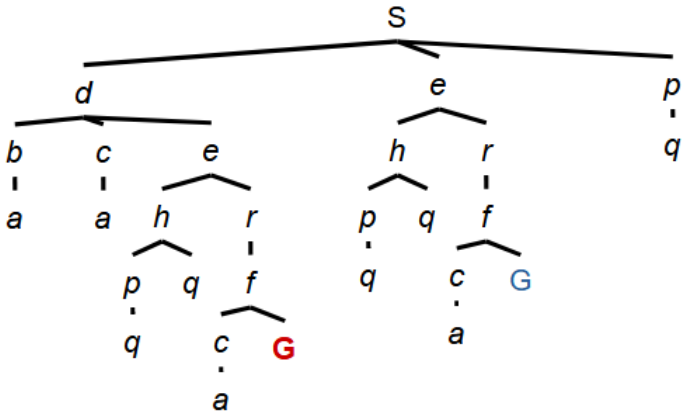
- Utilizaremos uma estrutura de pilha para implementar as visitas
- LIFO (Last-In First-out)

Implementação

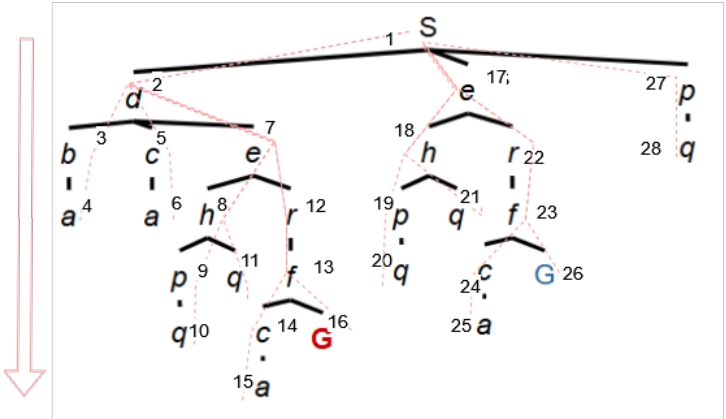
Temos três abordagens de visitas aos nós:

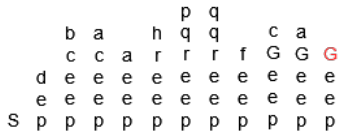
- Em ordem: Esquerda, Nó, Direita;
- Pré-ordem: Nós, Esquerda, Direita;
- Pós-ordem: Esquerda, Direita, Nó;

Implementação usando pilha



Implementação usando pilha - Referência



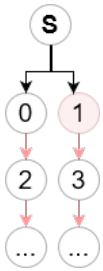

S d e b c e a c e a e e h r p q f c a G p p p

Características - Completude

- Se a solução existe, ela será encontrada, desde que a profundidade da árvore seja finita

Características - Completude

- Imaginem um problema que possua um caráter cíclico, ou que por sua natureza seja expansível infinitamente
- Exemplo, adivinhar um número (1); modelado como: buscarei pelos pares e depois pelos ímpares
- Como o nível dos números é infinito, a busca nunca irá terminar e atingir a solução
- É possível alterar a modelagem e resolver o problema com BP?



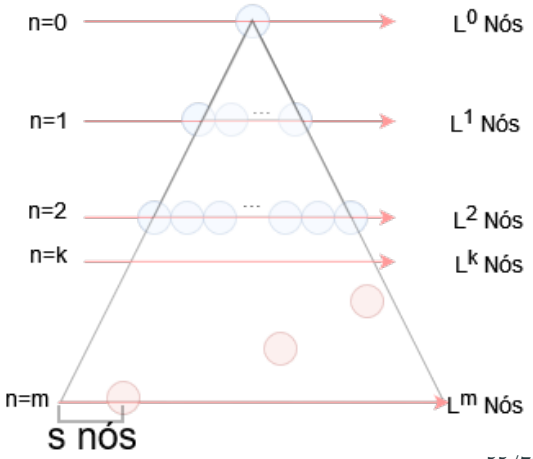
Características - Otimalidade

- Considerando que o nível da árvore está diretamente relacionado ao custo, tendo custos menores associados a níveis mais próximos da raiz da árvore
- A primeira solução encontrada nem sempre será ótima[1]

[1] vide exemplo de árvore

Características - Complexidade de Tempo

- Considerem o grau máximo de cada nó igual a L
- Considere que a solução mais a esquerda está a s nós a esquerda no último nível (m)
- A quantidade de nós em cada nível é igual a L^k sendo k o nível escolhido
- No pior caso a solução está no último nó do m-ésimo nível ($s = L^m$)
- $O(\sum_{i=0}^{m} L^i) = O(L^m)$



Características - Complexidade de Tempo

- Notem que no pior caso a busca irá percorrer toda a árvore
- Mas tende a operar melhor quando a solução está em nós-folha mais à esquerda

Características - Complexidade de Espaço

- Considerando que a árvore de decisão possui apenas os nós do caminho até a solução, inclusive os 'irmãos' dos nós nesse caminho
- Na pior das hipóteses temos m nós da raiz à folha e L nós em cada nível, portanto $O(L * m)$

Uso

- Exemplos de problemas que parecem exigir menos tentativas seguindo essa estratégia?

Busca em Profundidade Limitada (Iterativa)

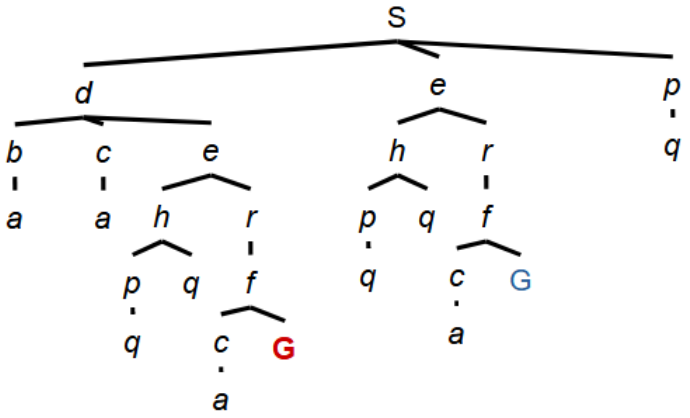
Motivação

- A busca em largura garante otimalidade
- A busca em profundidade armazena menos informação
- E se fizermos ambas?

2 versões

- Limitada (*limited depth search*): onde definimos um limite e não o alteramos
- Iterativa (*Iterative Depth Search*): onde o limite vai sendo aumentado iterativamente

Funcionamento



Implementação

- Limitaremos a profundidade da árvore e efetuaremos uma busca em profundidade normal
- Caso decidamos pela versão de profundidade iterativa, a profundidade é incrementada cada vez que todos os nós são visitados
- Isso gera uma complexidade extra de implementação e armazenamento para evitar visitas repetidas

Características - Completude

- Se a solução existe, ela será encontrada, desde que a profundidade da árvore seja finita e menor que a profundidade escolhida; sempre encontrará solução (existente) caso a profundidade seja iterativa.

Características - Otimalidade

- Considerando que o nível da árvore está diretamente relacionado ao custo, tendo custos menores associados a níveis mais próximos da raiz da árvore
- Similar à BP, mas suavizada pela limitação de profundidade

Características - Complexidade de Tempo

- Similar à Busca em Largura, se profundidade limite for igual à profundidade da solução
- O que inclui o processo iterativo onde o limite vai incrementando em 1
- E as visitas repetidas? Tendem a ser no topo da árvore onde poucos nós se concentram, mas deve ser avaliado e otimizado, se possível

Características - Complexidade de Tempo

- Notem que no pior caso a busca irá percorrer toda a árvore
- Mas tende a operar melhor quando a solução está em nós mais à esquerda

Características - Complexidade de Espaço

- Considerando que a árvore de decisão possui apenas os nós do caminho até a solução, inclusive os 'irmãos' dos nós nesse caminho
- Na pior das hipóteses temos m nós da raiz à folha e L nós em cada nível, portanto $O(L * m)$

Uso

- Exemplos de problemas que parecem exigir menos tentativas seguindo essa estratégia?

Sumário

Busca em	Completa?	Ótima?	Tempo	Espaço
Largura	Sim	Sim*	$O(L^s)$	$O(L^s)$
Profundidade	Sim	Não	$O(L^m)$	$O(L * m)$
Profundidade Limitada	Não	Não	$O(L^p)$	$O(L * p)$
Profundidade Iterativa	Sim	Sim**	$O(L^s)$**	$O(L * s)$**

Tabela 1: L = grau máximo da árvore; s = menor nível com solução; m = maior nível da árvore; p = profundidade limite escolhida; *Considerando que o custo é equivalente ou diretamente proporcional ao nível da árvore. **Considerando profundidade sendo iterada em 1